



BURSA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ

PROGRAMLAMA DİLLERİ
ERLANG PROGRAMLAMA DİLİ

23360859051

İSMAİL BİLGİN

2026

Erlang, 1980'lerin sonunda Ericsson çatısı altında Joe Armstrong, Robert Virding ve Mike Williams tarafından, özellikle yüksek erişilebilirlik (high availability) ve hata toleransı (fault tolerance) gerektiren karmaşık telekomünikasyon sistemlerini yönetmek amacıyla geliştirilmiş eşzamanlı (concurrent) ve fonksiyonel bir programlama dilidir.

1) Nesneye Yönelik Kavramları Yerine Getirme Biçimi

Erlang'da nesneler yerine tamamen izole çalışan süreçler (processes) bulunur. Kullanıcı verinin iç temsilini bilmek zorunda değildir. Her süreç kendi iç durumunu (state) saklar ve bu durum dışarıdan erişilemez.

Erlang'da süreçler birbirinin metodlarını çağırmak yerine sadece asenkron mesajlar ("!" operatörü) göndererek birbiriyle haberleşir.

Hangi metodun çalışacağını derleme zamanında değil çalışma zamanında (runtime) belirlenmesi kuralını, gelen mesajın türüne ve verisine göre ilgili fonksiyon bloğunu anında seçen **örüntü eşleştirme** ile gerçekleştirir.

Erlang dilinde sınıf hiyerarşisi olmadığı için çoklu mirasta ortaya çıkan "diamond problem" ve karmaşıklıklar Erlang'da yaşanmaz.

Erlang

```
% Bir sürece mesaj gönderme ve alma:  
SüreçPid ! {selam, self()},  
  
receive  
    {selam, Kimden} -> io:format("Selam alındı!")  
end.
```

2) Paralelliği Sağlama ve Uygulama Biçimi

Erlang, işletim sisteminin thread'lerini kullanmak yerine kendi sanal makinesi (BEAM) üzerinde çalışan çok hafif süreçler kullanarak alt program seviyesinde (unit-level) paralellik sağlar.

Her sürecin kendine has izole belleği olduğundan geleneksel dillerdeki paylaşılan kaynağa aynı anda erişimden doğan "Rekabet Senkronizasyonu" problemi çözülmüştür.

Süreçler arası iletişim tamamen asenkron mesaj gönderimi ile yapılır. Süreçler birbirlerinin posta kutularına veri yollarlar ve bu veriler sırasıyla receive bloğu ile işlenir.

3) Fonksiyonel Yapılara Uygunluk ve Nasıl Sağladığı

Erlang, değişkene bir kez değer atadığında bellekteki o değer değiştirilemez. Bu durum yan etkileri kaldırır (side effects) ve binlerce sürecin aynı anda veri üzerinde işlem yapmasını güvenli bir hale getirir. Bu değişmezliğe (Immutability) örnektir.

Değişkenlerin durumları değişmediği için “for” ve “while” gibi iteratif döngüler Erlang’da **yoktur**. Döngüsel işlemler fonksiyonların kendi kendini çağırması ile (Recursion) ile yapılır.

Fonksiyonlar tıpkı tamsayı veya karakterler gibi bir değişkene atanabilir ve başka bir fonksiyona argüman olarak atanabilir veya bir fonksiyondan sonuç olarak döndürülebilir.

“If else” blokları yerine fonksiyonlar aldıkları argümanların yapısına göre örtülü eşleştirme ile çalışır. Aynı isimde fakat farklı veri yapıları (tüple, liste vb.) bekleyen çok sayıda fonksiyon gövdesi yazılarak akış kontrol edilir.

Erlang

```
% Döngü yerine Özyineleme (Recursion) örneği:  
faktoriyel(0) -> 1;  
faktoriyel(N) -> N * faktoriyel(N - 1).
```

4) Binding ve Bellek Yönetimini Nasıl Sağladığı

Erlang’da değişkenler çalışma zamanında sadece bir kez değer bağlanır ve bu değer değiştirilemez. Ayrıca herhangi bir fonksiyon bloğunun çalıştırılacağına derleme zamanında değil çalışma zamanında gelen argümanların örüntüsüne (pattern matching) bakarak dinamik olarak karar verir.

Erlang’da her bir sürecin (process) tamamen kendisine ait, diğer süreçlerden yalıtılmış bağımsız bir Heap ve Stack alanı vardır.

Örtük Serbest Bırakma (Implicit Deallocation) konusunda ise programcı belleği **doğrudan yönetemez**, Erlang **Garbage Collection** kullanır. Bu sayede de Dangling Pointer gibi problemler tamamen ortadan kalkar.

Diğer dillerden (Java gibi) en önemli farklarından biri de çöp toplayıcının **tüm sistemi durdurmamasıdır**. Sadece belleği dolan sistemin kendi içindeki Heap alanı temizlenirken diğer süreçler çalışmaya devam eder.

5) Diğer Dillerden Ayrılan En Önemli Özelliği

Diğer diller hataları yakalamak için try-catch ve karmaşık kodlar yazarken Erlang’da böyle bir yazım söz konusu değildir. Eğer herhangi süreçte bir hata olursa sistem süreci çökmeye bırakır.

Çöken süreçler ise onları izleyen gözetmenler (Supervisors) tarafından anında algılanır ve otomatik olarak temiz bir durumda (state) yeniden başlatılır. Bu kendi kendini onaran yapı, Erlang sistemlerinin %99.999 (Nine Nines) oranında kesintisiz (uptime) çalışmasını sağlayan en eşsiz özelliktir.

6) Dil Sözdizimi (syntax), ya da Anlam (semantics) Olarak Sahip Olduğu Özellikler

Erlang, sözdizimi temelini Prolog'dan alır. İfadeler noktalı virgül (;) ile ayrılır ve fonksiyonlar veya ifadelerin sonu tıpkı normal bir cümle bitişi gibi nokta (.) ile sonlanır.

Erlang'da “ = ” işareti bir atama operatörü değil, bir örüntü eşleştirme (pattern matching) operatörüdür. Sağ taraf ile sol tarafın yapısal olarak eşleşip eşleşmediğini kontrol eder.

Kendi adından başka bir değeri olmayan, harfle başlayan ve küçük harfle yazılan sabit değerler vardır ve bunlara **atom** denir. “ok”, “error” ve “true” bunlara örnek olarak verilebilir. Semantik olarak programın durumlarını veya mesaj tiplerini ifade etmek için kullanılırlar.

Değişkenlerin tipleri derleme zamanında bildirilmez, çalışma zamanında belirlenir. Ancak Erlang, farklı tipleri (örneğin bir string ile bir integer) örtük olarak birbirine çevirmez (Strong Type) doğrudan hata fırlatır.

Erlang, Java veya C++'taki gibi derleme zamanı Generic tiplere (List<T> gibi) ihtiyaç duymaz. Dinamik tipleme yapısı nedeniyle bir liste veya tuple içerisine aynı anda her tür veri tipi (sayı, atom, başka bir liste veya süreç ID'si) eklenebilir. Bu doğal esneklik, statik dillerdeki generic yapıların sunduğu ihtiyacı doğrudan karşılar.

Sunum linki:
<https://youtu.be/ONeWTPJk6II>

Erlang

```
% '=' operatörü eşleştirme yapar, atomlar ve tuple kullanımı:  
{Durum, İsim} = {ok, "Ahmet"}.  
% Durum = ok, İsim = "Ahmet" olarak bağlanır.  
  
{Hata, _} = {error, "Bağlantı koptu"}.
```

7) ADT Yapısı, Tipleri ve Generic Yapısı

Erlang'da özel bir ADT tanımlamak için Tuple “ {...} ” ve List “ [...] ” veya Map “ #{...} ” yapıları birleştirilerek kullanılır. Kayıtlar (records), altta yatan bir tuple yapısının isimlendirilmiş halidir ve yapılandırılmış veriyi (ADT) temsil etmek için kullanılır.